

Open Agent Trust Stack (OATS): A System Specification for Zero-Trust AI Agent Execution

Jascha Wanger
ThirdKey AI
jascha@thirdkey.ai

Version 1.3.0 — 19 May 2026

Specification home: openagenttruststack.org

Zenodo preprint: [10.5281/zenodo.20298543](https://zenodo.org/record/20298543)

Abstract

As AI systems evolve from assistants into autonomous agents executing consequential actions, the security boundary shifts from model outputs to tool execution. Traditional security paradigms — log aggregation, perimeter defense, post-hoc forensics, and runtime interception of fully-formed actions — cannot adequately protect systems where AI-driven actions are irreversible, execute at machine speed, and originate from potentially compromised orchestration layers. This paper introduces the Open Agent Trust Stack (OATS), an open specification for zero-trust AI agent execution. OATS is built on three architectural convictions. First, *allow-list enforcement*: rather than intercepting arbitrary actions and deciding which to block, OATS constrains what actions can be expressed through declarative tool contracts, making dangerous actions structurally inexpressible. Second, *compile-time enforcement*: the Observe–Reason–Gate–Act (ORGA) reasoning loop uses typestate programming so that skipping the policy gate is a type error, not a runtime bug. Third, *structural independence*: the Gate phase is architecturally isolated from LLM influence. OATS specifies five layers: (1) the ORGA reasoning loop with compile-time phase enforcement, (2) declarative tool contracts with typed parameter validation, (3) a cryptographic identity stack providing bidirectional trust between agents and tools, (4) a formally verifiable policy engine operating on structured inputs, and (5) hash-chained cryptographic audit journals with Ed25519 signatures for tamper-evident forensic reconstruction. OATS is model-agnostic, framework-agnostic, and vendor-neutral. The architecture is informed by operational experience with a production runtime (Symbiont) [26] that has operated autonomously for approximately nine months. Initial empirical results validating five of seven core conformance requirements are now available through three companion preprints [27, 23, 24] and the `symbiont-orga-demo` reference corpus [25]; results are summarized in Section 14. The specification continues to stand independently of any single implementation, and remaining empirical work is identified as future deliverables.

1 Introduction

1.1 The Runtime Security Gap

AI agents now execute consequential actions across enterprise systems: querying databases, sending communications, modifying files, invoking cloud services, and managing credentials. Through function calling, plugins, external APIs, and protocol-based tool servers such as the Model Context Protocol (MCP) [2], these agents perform multi-step tasks without human intervention [21, 30].

These actions exhibit five characteristics that existing security paradigms cannot adequately address:

- **Irreversibility.** Tool executions produce immediate and often permanent effects: database mutations, financial transactions, credential changes, or data exfiltration [29, 31]. Once executed, the damage is done.
- **Speed.** Agents execute hundreds of tool calls per minute, far exceeding human capacity for real-time review [19]. Multi-step attack chains complete within seconds [4].
- **Compositional risk.** Individual actions may each satisfy policy while their composition constitutes a violation [18]. Reading a confidential file is permitted; sending email is permitted; doing both in sequence may constitute exfiltration.
- **Untrusted orchestration.** Prompt injection and indirect instruction attacks mean the model’s apparent intent cannot be trusted [15, 9]. Adversarial prompts can be embedded in documents, emails, and images that agents process [7].
- **Privilege amplification.** Agents routinely operate under static, high-privilege identities misaligned with the principle of least privilege [11].

The gap in the current security landscape lies at the intersection of prevention and context-awareness: no existing system can block actions before execution based on both static policy and accumulated session context while simultaneously constraining what actions can be expressed in the first place. This is the gap that OATS addresses.

This paper makes two contributions: a normative system specification defining the runtime enforcement boundary for autonomous agent execution, and an implementation-grounded evaluation methodology derived from operational experience with a production runtime. The specification is the primary artifact; the evaluation framework (Section 14) defines the criteria; initial empirical results from a reference corpus are reported in Section 14.7. The contribution is a new runtime security abstraction with testable conformance properties, accompanied by demonstrated implementation feasibility for a substantial subset of those properties.

1.2 Design Principles

OATS is built on three architectural convictions, each addressing a structural weakness in current approaches.

Allow-list over deny-list. Current runtime security approaches operate on a deny-list model: the agent formulates an action, a security system intercepts it, evaluates it, and decides whether to allow or block. This requires enumerating dangerous behavior — an enumeration that is incomplete by definition. OATS inverts this model. The agent fills typed parameters defined by a declarative tool contract. The runtime validates parameters against the contract, constructs the invocation from a template, and executes. The agent never generates raw commands or constructs unconstrained API calls. Within the scope of contracted tools, dangerous actions cannot be expressed because the interface does not permit them. Actions that bypass the contract layer entirely (e.g., direct network calls from compromised agent code) require complementary controls such as sandboxing (Section 10).

Compile-time over runtime enforcement. When enforcement correctness is verified only at runtime, a code change that introduces a path bypassing the policy engine goes undetected until that path is exercised. OATS addresses this through the Observe–Reason–Gate–Act (ORGA)

cycle, which uses type-level programming (tpestates) so that skipping the Gate phase, dispatching tools without reasoning first, or observing results without dispatching are compile-time errors. In a correctly implemented tpestate, the type system enforces that every action passes through policy evaluation. This property holds for code paths within the tpestate-governed loop; it does not extend to code that circumvents the loop entirely, which is why sandboxing and network isolation provide complementary enforcement.

Structural independence over trust assumptions. When the policy engine shares context, memory, and execution environment with the orchestration layer it governs, an LLM compromised through prompt injection can potentially influence the evaluation of its own actions. In OATS, the Gate phase receives a structured action proposal and evaluates it against policy using a formally verifiable policy engine. The LLM cannot modify, bypass, or influence the Gate’s evaluation.

1.3 Contributions

This specification makes six contributions:

- **Tpestate-enforced reasoning loop.** We specify the ORGA cycle with compile-time phase enforcement, designed to prevent policy evaluation from being skipped, circumvented, or reordered within the loop (Section 5).
- **Allow-list tool contracts.** We specify a declarative tool contract format that constrains agent–tool interaction to typed, validated parameters, making dangerous actions structurally inexpressible (Section 6).
- **Layered cryptographic identity.** We specify a bidirectional identity stack providing mutual authentication between agents and tools via domain-anchored cryptographic verification, with explicit cryptographic-agility constraints (Section 7).
- **Hash-chained audit journals.** We specify cryptographically signed, hash-chained event journals for tamper-evident forensic reconstruction (Section 9).
- **Conformance requirements.** We define minimum requirements for OATS-compliant systems, enabling objective evaluation of implementations (Section 12).
- **Initial empirical evaluation.** We report initial results from the `sybiont-orga-demo` reference corpus [25], which validates five of seven core conformance requirements through three companion preprints [27, 23, 24]. The corpus measures attack-suite refusal rates, fence non-redundancy via stack-stripping ablation, runtime overhead across nine widely available hosted LLMs, and — via the substrate-comparison preprint [24] — the marginal contribution of structural enforcement relative to OS-isolation alternatives (Section 14.7).

OATS’s novelty is not any single component in isolation — tpestates, policy engines, cryptographic signatures, audit logs, and sandboxing each have extensive prior art. The contribution is the integration of five layers into a unified runtime security model centered on consequential action execution, with three properties not found in prior work in combination: (a) expressibility constraints that eliminate action categories before policy evaluation, (b) compile-time enforcement that the policy gate executes on every dispatch path within the loop, and (c) bidirectional cryptographic identity binding actions to verified agents and verified tools. The conformance requirements formalize these properties into testable criteria, enabling objective comparison across implementations. Initial empirical results (Section 14.7) demonstrate that these properties are achievable in practice, and

that they produce measurable defense beyond OS-isolation baselines on four of five action vectors evaluated [24].

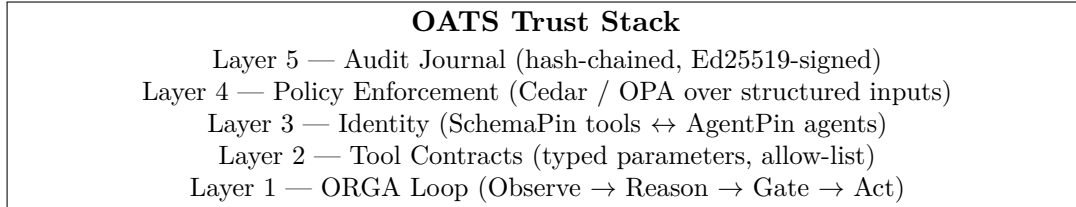


Figure 1: OATS trust stack. Five layers, each addressing a distinct security question. The ORGA loop (Layer 1) enforces that the Gate runs; tool contracts (Layer 2) constrain expressible actions; the identity stack (Layer 3) provides mutual authentication; the policy engine (Layer 4) evaluates authorization; the audit journal (Layer 5) records decisions.

1.4 Document Structure

Section 2 reviews related work. Section 3 formalizes the problem and defines the system model. Section 4 characterizes the threat landscape. Section 5 presents the ORGA loop architecture. Section 6 specifies the tool contract layer. Section 7 specifies the identity layer. Section 8 specifies the policy enforcement layer. Section 9 specifies the audit layer. Section 10 addresses sandboxing and isolation. Section 11 addresses inter-agent communication. Section 12 defines conformance requirements. Section 13 presents implementation architectures. Section 14 defines the evaluation framework, with Section 14.7 reporting initial empirical results from the reference corpus. Section 15 identifies known limitations. Section 16 identifies open research directions. Section 17 concludes.

2 Related Work

2.1 Agent Security Research

The security risks of LLM-based agents have been catalogued by several surveys. Ruan et al. [18] and Wu et al. [29] provide comprehensive threat taxonomies covering prompt injection, tool misuse, and data exfiltration. Su et al. [19] focus on autonomy-induced risks including memory poisoning and deferred decision hazards. Debenedetti et al. [4] introduce AgentDojo for evaluating attacks and defenses against LLM agents, while Ye et al. [31] propose ToolEmu for identifying risky agent failures. These works characterize the problem space but do not propose runtime enforcement architectures. OATS builds on their threat models and contributes a system specification for constraining and evaluating actions before execution.

Gaire et al. [6] systematize security and safety risks in the Model Context Protocol ecosystem, providing a taxonomy of threats to MCP primitives. Their analysis of tool poisoning and indirect prompt injection directly informs OATS’s threat model for tool supply chain attacks.

2.2 Runtime Security Specifications

Errico [5] introduces Autonomous Action Runtime Management (AARM), a system specification for securing AI-driven actions at runtime. AARM formalizes the runtime security gap, proposes an action classification framework distinguishing forbidden, context-dependent deny, context-dependent allow, and context-dependent defer actions, and specifies conformance requirements for pre-execution

interception, context accumulation, policy evaluation, and tamper-evident receipts. OATS shares AARM’s identification of the action layer as the stable security boundary and incorporates its context-dependent action classification. OATS extends this foundation with compile-time enforcement of the reasoning loop, allow-list tool contracts, concrete cryptographic identity protocols, and multi-tier execution isolation.

2.3 Industry Frameworks

Google’s Cloud CISO perspective [3] advocates defense-in-depth and runtime controls for agents. AWS’s Agentic AI Security Scoping Matrix [17] provides a risk assessment framework complementing runtime enforcement with deployment-time scoping. Microsoft’s governance framework [10] addresses organizational controls including identity management and approval workflows. Raza et al. [16] present a TRiSM framework for agentic multi-agent systems. These frameworks provide lifecycle governance; OATS focuses specifically on the runtime enforcement layer.

2.4 Access Control and Policy Languages

OATS’s policy enforcement layer builds on established access control research. RBAC and ABAC evaluate permissions against static attributes but lack session-level context accumulation [11]. Capability-based security [8] constrains authority propagation but does not address compositional risks of non-deterministic agents. Policy languages such as OPA [13] and Cedar [1] provide expressive evaluation engines suitable as backends for OATS’s policy evaluation component. AWS’s independent adoption of Cedar for AgentCore provides supporting evidence for the thesis that formal policy languages belong at the agent execution boundary.

2.5 Complementary Standards

OATS is complementary to several existing standards. The OWASP Top 10 for LLM Applications [14] catalogs vulnerabilities; OATS provides runtime enforcement mitigating tool misuse, excessive agency, and insecure output handling. The NIST AI RMF [12] provides a risk management framework; OATS provides technical enforcement implementing portions around governance, monitoring, and accountability. MCP [2] defines agent-tool communication; OATS defines how to govern actions flowing through that (or any) tool invocation mechanism.

2.6 Comparative Positioning

Table 1 positions OATS relative to existing approaches across the security properties specified in this paper. Checkmarks indicate the property is addressed by the approach; dashes indicate it is not.

3 Problem Formalization

3.1 System Model

An autonomous agent execution system consists of: a set A of possible actions a tool can execute; a set T of typed tool contracts $\kappa \in T$ constraining how A is invoked; an LLM orchestrator O proposing actions based on user requests and accumulated context; a policy engine P evaluating action proposals against organizational policy; a context store C accumulating session state; an

Property	Prompt guard.	Deny-list	Sandbox-only	AARM [5]	OATS
Constrains expressible actions	–	–	–	–	✓
Pre-execution policy gate	–	✓	–	✓	✓
Session context accumulation	–	–	–	✓	✓
Cryptographic tool identity	–	–	–	–	✓
Cryptographic agent identity	–	–	–	–	✓
Tamper-evident audit journal	–	–	–	✓	✓
Compile-time phase enforcement	–	–	–	–	✓
Gate independent of LLM	–	Partial	N/A	Partial	✓
Execution isolation	–	–	✓	–	✓
Formal conformance criteria	–	–	–	✓	✓

Table 1: Comparative positioning. “Partial” indicates the property is architecturally possible but not structurally enforced by the specification. AgentDojo [4] and ToolEmu [31] are evaluation frameworks rather than enforcement runtimes and are not included; they are complementary to OATS rather than alternatives.

identity verifier I authenticating agents and tools; an audit journal J recording all decisions and outcomes; and an executor E dispatching approved actions and capturing evidence.

3.2 Action Authorization

For each proposed action $a \in A$ in session context C under agent identity I , the runtime MUST evaluate whether a is authorized. Authorization requires:

- **Structural validity.** a conforms to a declared tool contract $\kappa \in T$.
- **Policy compliance.** a satisfies organizational policy Π given context C and identity I .
- **Context-aware evaluation.** a is evaluated against both static policy and accumulated session context.
- **Identity verification.** Both the agent invoking a tool and the tool being invoked are cryptographically verified.
- **Forensic completeness.** Every action, its context, the policy decision, and the execution outcome are recorded in a tamper-evident journal.

4 Threat Model

OATS operates on a fundamental assumption: the AI orchestration layer O cannot be trusted as a security boundary. The model processes untrusted inputs through opaque reasoning, producing actions that may serve attacker goals rather than user intent.

4.1 Threat Summary

4.2 Attack Lifecycle

Attacks against AI agents typically follow a four-stage lifecycle: (1) *injection* — attacker embeds malicious instructions in content the agent processes; (2) *hijacking* — the agent interprets malicious content as legitimate instructions; (3) *execution* — the agent invokes tools with attacker-controlled

Threat	Attack vector	OATS control
Prompt injection	User input, documents, tool outputs, images	Tool contracts (structural), policy enforcement, context-dependent deny
Malicious tool outputs	Adversarial tool responses	Post-tool action restrictions, context tracking, output schema validation, content sanitization (§6.6)
Confused deputy	Ambiguous/malicious instructions	Bidirectional identity verification, step-up approval, intent alignment
Over-privileged credentials	Excessive token scopes	Least-privilege enforcement, scoped credentials
Data exfiltration	Action composition	Context accumulation, compositional policies, scope enforcement
Goal hijacking	Injected objectives	Action-level policy, semantic distance tracking
Intent drift	Agent reasoning divergence	Context accumulation, semantic distance threshold, deferral
Memory poisoning	Persistent context manipulation	Provenance tracking, anomaly detection, journal comparison
Tool supply chain	Tampered contracts, spoofed tools	Cryptographic tool integrity verification, TOFU pinning, algorithm allowlisting (§7.5)
Cross-agent propagation	Multi-agent delegation	Cross-agent context, transitive trust limits, blast-radius containment
Orchestrator-injection	Adversarial tool outputs repackaged into trusted orchestration arguments	Agent-summary typed parameters with content validation (§6.6)

Table 2: Threat summary and OATS mitigations.

parameters; (4) *impact* — actions produce irreversible effects. OATS intervenes at two points: between stages 2 and 3 (the Gate blocks actions that violate policy), and before stage 3 begins (tool contracts constrain what parameters the agent can express).

Intent drift follows a different lifecycle without explicit injection. The agent’s reasoning gradually diverges through plausible-seeming steps. OATS detects this through context accumulation and semantic distance tracking regardless of cause.

4.3 Trust Assumptions

Trusted: the OATS runtime (ORGA loop, policy engine, tool contract executor, journal, identity verifier); cryptographic primitives and key management; the policy store and authoring process; the underlying infrastructure (OS, network, hardware); the compiler and type system.

Untrusted: the AI model and its outputs; the orchestration layer; user inputs and prompts; tool outputs and retrieved data; external documents, emails, web content, and multi-modal inputs; agent memory and conversation history; tool contracts from unverified publishers.

Partially trusted: tool implementations (OATS constrains invocation but cannot prevent internal bugs); human approvers (OATS routes step-up authorization but cannot prevent social engineering); verified tool contracts (verified as untampered, but the tool may have vulnerabilities).

4.4 Out of Scope

OATS addresses runtime action security. The following threats require complementary controls: model training data poisoning or weight manipulation (pre-deployment ML security); denial of service against the OATS runtime (infrastructure availability); physical or infrastructure-level attacks (physical security); social engineering of human approvers (security awareness training); code-level vulnerabilities within tool implementations (application security testing); memory storage security (separate storage controls). OATS is one layer in a defense-in-depth strategy.

5 Core Architecture: The ORGA Loop

The ORGA (Observe–Reason–Gate–Act) loop is the core execution engine for OATS-compliant runtimes. It drives a multi-turn cycle between an LLM, a policy gate, and external tools through four mandatory phases.

5.1 Phase Definitions

Observe. Collect results from previous tool executions. Incorporate tool outputs, error messages, policy denial feedback, and environmental signals into the agent’s context. This phase also integrates knowledge retrieval (RAG-enhanced context from vector-backed storage) when available.

Reason. The LLM processes accumulated context and produces proposed actions (tool calls or text responses). The LLM sees tool definitions derived from tool contracts but never sees raw invocation details. The LLM’s output is a structured proposal, not an executable action.

Gate. The policy engine evaluates each proposed action. This phase operates entirely outside LLM influence. The Gate receives the proposed action, the accumulated session context, and the agent’s identity, and evaluates them against organizational policy. The Gate produces one of five decisions: Allow, Deny, Modify, Step-Up, or Defer. Denial reasons are recorded in the audit journal and fed back to the LLM as observations in the next Observe phase.

Act. Approved actions are dispatched to tool executors. The tool contract executor validates parameters against the contract’s type system, constructs the invocation from the contract’s template, executes with timeout enforcement, captures output in a structured evidence envelope, and records the execution in the audit journal.

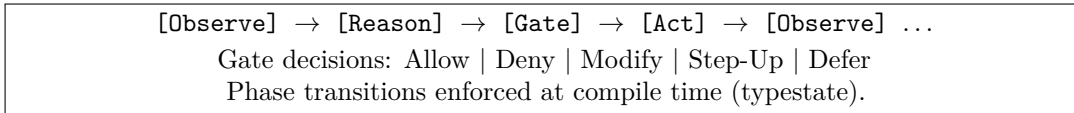


Figure 2: ORGA loop with tystate-enforced phase transitions. The Gate phase is mandatory and structurally independent of the LLM. Denied actions produce feedback observations; approved actions proceed to tool execution.

5.2 Tystate Enforcement

Phase transitions MUST be enforced at compile time. Each phase is a distinct type. The loop state machine can only call methods appropriate to its current phase:

```
AgentLoop<Reasoning> --produce_output()--> AgentLoop<PolicyCheck>  
AgentLoop<PolicyCheck> --check_policy()----> AgentLoop<ToolDispatching>  
AgentLoop<ToolDispatching>--dispatch()-----> AgentLoop<Observing>
```



```
AgentLoop<Observing> --observe()-----> AgentLoop<Reasoning> | LoopResult
```

The following are compile-time errors, not runtime bugs:

- Transitioning from Reasoning to ToolDispatching (skipping policy check)
- Transitioning from PolicyCheck to Observing (dispatching without execution)
- Transitioning from Reasoning to Observing (skipping both Gate and Act)

Implementations in languages without native typestate support MUST provide equivalent enforcement through runtime checks with 100% path coverage testing and documented verification that all tool dispatch paths pass through the Gate. Such implementations SHOULD acknowledge that runtime enforcement provides weaker assurance than compile-time enforcement and document the residual risk.

5.3 Dynamic Branching and Termination

The only dynamic branch in the ORGA loop is after Observe: the loop either continues (returning to Reason) or completes (producing a final result). This is a standard pattern match on a concrete type, not dynamic dispatch. All other transitions are strictly linear.

The loop terminates when the LLM produces a final text response, iteration limits are reached, token or time budgets are exhausted, or a circuit breaker trips.

5.4 Policy Denial Feedback

When the Gate denies an action, the denial reason MUST be fed back to the LLM as an observation. This allows the LLM to adjust its approach. The Gate evaluates each subsequent proposal independently; denials are not negotiable.

5.5 Scope of Assurance

Typestate enforcement provides a specific, bounded property. To prevent overinterpretation, we state exactly what is and is not covered.

What typestate enforcement covers. Within the ORGA loop, the type system enforces that every transition from action proposal to tool dispatch passes through the Gate phase. In a Rust implementation, this is a compile-time property: any code path that attempts `AgentLoop<Reasoning>` $\rightarrow$ `AgentLoop<ToolDispatching>` without consuming an intermediate `AgentLoop<PolicyCheck>` is rejected by the compiler. A proof sketch follows.

Proof sketch. Let R , P , D , O denote the Reasoning, PolicyCheck, ToolDispatching, and Observing phases. Each phase is a distinct zero-sized type. The only method consuming R produces P ; the only method consuming P produces D ; the only method consuming D produces O ; and O produces either R (continue) or a terminal value (complete). Because each method takes `self` by value (consuming the prior state), no valid Rust program can hold two phase values simultaneously or skip a phase. The compiler’s ownership and move semantics enforce this without runtime checks. This argument depends on the type signatures being correctly declared; it does not require trust in runtime behavior. This property has been empirically validated through nine compile-fail tests on the reference implementation [27].

What typestate enforcement does not cover. The property applies only to code paths mediated by the `AgentLoop` runner. It does not provide whole-program non-bypass assurance. Specifically: (a) agent code that invokes tools through a separate code path not mediated by the ORGA runner is unconstrained by the typestate; (b) plugins, FFI calls, or dynamically loaded code may bypass the loop; (c) network-level tool invocations from within the sandbox are not mediated by the type system. These residual risks are addressed by sandboxing (Section 10) and network isolation as defense-in-depth layers, not by the typestate itself. An OATS-compliant deployment SHOULD combine typestate enforcement with at least one complementary isolation mechanism.

5.6 Gate Independence

The Gate phase is designed to operate outside LLM influence. This section specifies what “outside LLM influence” means concretely and what an implementation must demonstrate.

Structural requirements. The Gate MUST receive input as a typed, serialized data structure (e.g., a Rust struct, a JSON object conforming to a fixed schema) containing: tool name, operation, validated parameters, agent identity, and accumulated session context. The Gate MUST NOT receive natural language strings, LLM reasoning traces, or any content that requires language interpretation. The Gate MUST NOT share mutable memory, mutable references, or writable state with the LLM inference component. The Gate MUST NOT expose a callback, hook, or API that the LLM can invoke to modify Gate behavior during evaluation.

Implementation patterns. Conformant implementations may achieve Gate independence through any of the following mechanisms, listed in decreasing order of isolation strength: (a) separate process with IPC serialization boundary; (b) separate thread with immutable message passing and no shared mutable state; (c) synchronous function call with typed struct input, no closures capturing LLM state, and no interior mutability accessible from the LLM component. Pattern (a) provides the strongest isolation. Pattern (c) is acceptable when the implementation can demonstrate (via code review or static analysis) that no shared mutable path exists.

Verification. Conformance requirement C6 (Section 12) defines the verification procedure: inspect the Gate implementation, verify inputs are typed structs, verify no shared mutable references, verify no dynamic code paths parameterized by LLM output.

6 Tool Contract Layer

6.1 The Allow-List Principle

OATS inverts the conventional sandbox model:

Deny-list: agent constructs any action $\rightarrow$ runtime intercepts $\rightarrow$ deny if matches rule.
Allow-list (OATS): agent fills typed parameters $\rightarrow$ runtime validates against contract $\rightarrow$ executes from template.

Figure 3: Deny-list vs. allow-list enforcement models. In the allow-list model, the space of expressible actions is constrained by the tool contract before any policy evaluation occurs.

6.2 Contract Requirements

A tool contract κ MUST define:

- **Typed parameters.** Each parameter has a declared type from T with validation constraints. All string-based types MUST reject shell metacharacters (`;` `|` `&` `$` `\` `(` `)` `{` `}` `[` `]` `<` `>` `'`) by default.
- **Invocation mechanism.** Command template, HTTP request template, protocol server address, or interactive session definition. The LLM never constructs invocation details.
- **Output schema.** Expected structure of tool output. The executor validates parsed output before returning results to the agent.
- **Policy metadata.** Policy resource and action declarations enabling authorization without parsing tool-specific details.
- **Risk tier.** Risk classification (low, medium, high, critical) informing default policy generation and step-up thresholds.

6.3 Execution Modes

Tool contracts SHOULD support three execution modes sharing a common governance layer:

Mode	Description	Governance
Oneshot	Single invocation, return results	Per-invocation Gate evaluation
Session	Running process (PTY), per-interaction validation	Per-interaction Gate evaluation
Browser	Governed browser (CDP/Playwright), scoped navigation	Per-action Gate evaluation

Table 3: Tool contract execution modes.

6.4 Contract Integrity

Tool contracts MUST support cryptographic integrity verification. Signatures MUST cover the entire contract — parameters, validation rules, invocation templates, output schemas, and scope constraints. A contract failing verification MUST be rejected.

6.5 Schema Generation

Tool contracts SHOULD support automatic generation of protocol-compatible schemas (e.g., MCP `inputSchema` and `outputSchema`) from the contract definition. The LLM understands tool capabilities through generated schemas without the contract format being exposed.

6.6 Content Sanitization

New in v1.3.0.

Argument validation against shell metacharacters (§6.2) defends action-shape attacks but does not address all *content-shape* attacks: string parameters that pass type validation while carrying invisible or visually-deceptive characters intended to alter the semantics of downstream prompts, journals, or human review. Empirical evidence from the reference implementation [27, §6.8][23, §5] and the substrate-comparison sweep [24, §6.2] establishes that content-shape attacks require a dedicated fence; structural action fences cannot defend them.

Sanitization requirement (SHOULD). Tool contract executors and journal writers SHOULD sanitize agent-influenced string fields before they reach the agent’s reasoning context, the journal, or downstream tools. At minimum, sanitization SHOULD remove:

- ASCII control characters (C0 / DEL except `\t`, `\n`, `\r` where context-appropriate)
- C1 controls (U+0080..=U+009F)
- Zero-width characters (U+200B..=U+200F, U+2060..=U+2064)
- Bidi overrides (U+202A..=U+202E, U+2066..=U+2069)
- Word-joiner / invisible-operator block (U+2060..=U+2064)
- Byte-order marks (U+FEFF)
- Variation selectors (U+FE00..=U+FE0F, U+E0100..=U+E01EF)
- Unicode Tag block (U+E0000..=U+E007F)
- Soft hyphen (U+00AD)

NFKC normalization (SHOULD). Sanitization SHOULD apply NFKC normalization to defend against fullwidth and math-alphanumeric homoglyph bypasses, and SHOULD flag scripts mixed in unusual combinations (e.g., Latin + Cyrillic identifiers) for elevated scrutiny.

Trusted vs. untrusted strings. Sanitization applies to *agent-influenced* fields — strings whose value can be controlled by the LLM or by content the LLM has read. Runtime-internal strings (e.g., observations the runtime itself constructs, identity tokens, journal sequence numbers) are out of scope.

Reference implementation. The Symbiont reference implementation provides this functionality via the `sympi-invis-strip` crate (v0.3.0 as of v1.14.0), including a `sanitize_field_with_markup` variant that additionally strips HTML comment blocks and triple-backtick fenced blocks on surfaces where renderer-hidden markup has no legitimate use.

Scope and limits. Content sanitization is a defense-in-depth fence, not a complete defense. Section 15 notes the known ceiling against capable models that can construct content-shape attacks within the sanitized character set. Sanitization MUST NOT be the *only* defense against content-shape attacks; it complements policy evaluation, output schema validation, and (where applicable) downstream task graders.

7 Identity Layer

7.1 The Identity Problem

When AI agents interact with tools, services, and other agents, identity is typically self-asserted. An agent claims to be “Scout v2 from Tarnover LLC” with no cryptographic proof. A tool claims to offer a specific interface with no integrity verification. Self-asserted identity provides no security guarantee.

OATS specifies a two-layer cryptographic identity stack:

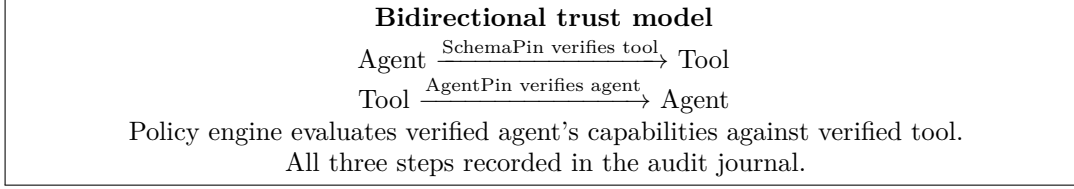


Figure 4: Bidirectional trust model. The agent verifies tool integrity (SchemaPin), the tool verifies agent identity (AgentPin), and the policy engine evaluates authorization. All three steps are recorded in the cryptographic audit journal.

7.2 Tool Integrity Verification

An OATS-compliant runtime **MUST** support cryptographic verification of tool contracts. The protocol provides:

- **Domain-anchored discovery.** Publishers host public keys at `/.well-known/` URIs (RFC 8615). No centralized registry.
- **Signature verification.** Contracts signed with ECDSA P-256 (or equivalent). Runtime verifies before registration.
- **TOFU key pinning.** On first encounter, the runtime pins the publisher's key. Subsequent key changes require explicit trust decisions.
- **Revocation support.** Publishers can revoke keys and schemas. Runtime checks revocation status.

7.3 Agent Identity Verification

An OATS-compliant runtime **SHOULD** support cryptographic agent identity verification:

- **Domain-anchored identity.** Organizations publish verifiable agent identity documents at `/.well-known/agent-identity.json`.
- **Short-lived credentials.** ES256 JWTs declaring identity, capabilities, and delegation chain.
- **Multi-step verification.** Signature verification, domain binding, capability validation, revocation checking, TOFU key pinning.
- **Delegation chains.** Maker-deployer delegation where the software builder and instance deployer are independently verifiable.
- **Capability scoping.** Credentials declare specific capabilities (e.g., `read:data`, `write:reports`).

7.4 Bidirectional Trust

The combined verification flow:

- Agent's runtime verifies tool contract integrity (tool has not been tampered with).
- Tool's runtime verifies agent identity (agent is authorized to act).
- Policy engine evaluates whether the verified agent's capabilities authorize use of the verified tool.
- Both verifications and the policy decision are recorded in the audit journal.

7.5 Cryptographic Agility and Algorithm Allowlisting

New in v1.3.0.

Self-describing credential formats (JWTs, signed manifests, X.509 chains) routinely advertise the cryptographic algorithm used to sign them. An implementation that trusts the advertised algorithm without restriction inherits the vulnerability surface of every algorithm the verifier library happens to support, including algorithms with known weaknesses or library-side vulnerabilities (e.g., RUSTSEC-2023-0071 reachable through `jsonwebtoken` v10 on certain RSA paths).

Allowlist requirement (SHOULD). Identity verifiers in an OATS-compliant runtime SHOULD enforce a per-credential-class algorithm allowlist that rejects unlisted algorithms at the header-inspection guard *and* at the verification API. Verifiers SHOULD NOT rely solely on validation-side filtering, since that path can be reached only after header inspection has already accepted the credential class.

Default allowlists. For each credential class OATS specifies:

Credential class	Allowed algorithms	Refused
Asymmetric Bearer tokens (AgentPin, agent-to-agent JWTs)	ES256, EdDSA	RS*, PS*, none, HS*
HMAC webhook / channel signatures	HS256	All asymmetric algorithms, none
Tool contract signatures (SchemaPin)	ECDSA P-256	All other algorithms
Audit journal entry signatures	Ed25519 (preferred) or ECDSA P-256	All other algorithms

Table 4: Per-credential-class algorithm allowlists.

Implementations MAY support additional algorithms when documented, when the deployment context requires interoperability with an external system that mandates the algorithm (e.g., Microsoft Teams Bot Framework requires RS256 for inbound token validation), and when the trust scope of those tokens is bounded to that external system. Implementations MUST NOT silently fall back to refused algorithms.

Aud / iss / exp validation. All JWT-shaped credentials MUST be validated for `aud`, `iss`, and `exp` claims. Verifiers MUST NOT provide an environment-variable or runtime escape hatch to disable audience validation; deployment requirements that conflict with `aud` validation indicate a misconfiguration, not a need for a bypass.

Verification (E1 / E2). Conformance verification (Section 12) inspects the runtime’s verifier configuration, attempts a signed token under each refused algorithm, and confirms refusal both at header inspection and at the algorithms field of the verification call.

8 Policy Enforcement Layer

8.1 Policy Engine Requirements

An OATS-compliant policy engine evaluates the tuple (a, C, I) and produces an authorization decision. The engine:

- MUST support five decisions: Allow, Deny, Modify, Step-Up, Defer.
- MUST evaluate both static policy and accumulated session context.
- MUST operate outside LLM influence: structured inputs, structured decisions, no natural language processing, no shared mutable state with the LLM.
- SHOULD support a formally verifiable policy language (Cedar [1], OPA [13], or equivalent).
- MUST default to deny.
- MUST default to *fail-closed construction*: when the runtime is started without an explicitly wired policy backend, the default behavior MUST be to deny every `ToolCall` and `Delegate` action with an explicit reason, not to silently allow them. A permissive mode MAY be provided for local development but MUST require an explicit opt-in (CLI flag or environment variable) and MUST emit a visible warning on every evaluated action. The distinction between spec-level “default to deny” (a policy-evaluation property) and operational “fail-closed construction” (a runtime-initialization property) is intentional; both are required.

8.2 Action Classification

OATS classifies actions into five categories based on how they should be evaluated:

Category	How identified	Evaluation	Decision
Structurally forbidden	Cannot be expressed via tool contract	None needed	N/A (inexpressible)
Policy-forbidden	Static policy match	Static policy only	DENY
Context-dep. deny	Policy allows, context mis-aligns	Static + context	DENY
Context-dep. allow	Policy denies, context aligns	Static + context	STEP-UP / ALLOW
Context-dep. defer	Insufficient/conflicting context	Indeterminate	DEFER

Table 5: Action classification framework. The “structurally forbidden” category exists only in systems with allow-list tool contracts.

8.3 Context Accumulation

The runtime MUST accumulate session context as an append-only, hash-chained log:

- **Original request.** The user’s initial instruction establishing intent.
- **Action history.** Sequence of actions proposed, approved, denied, deferred, and executed.

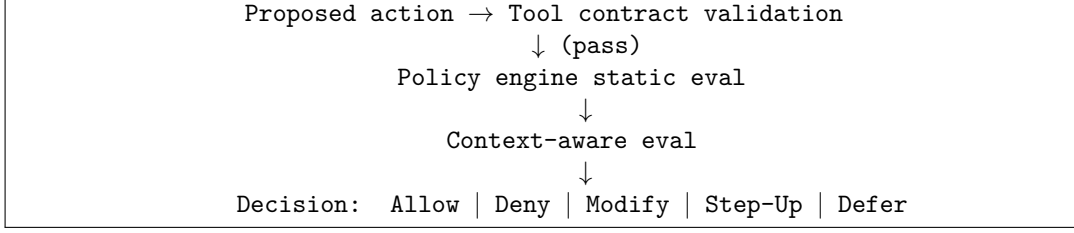


Figure 5: Action classification pipeline. Tool contract validation eliminates structurally forbidden actions before the policy engine is reached. Context-aware evaluation handles the remaining categories.

- **Data classification.** Sensitivity of information accessed. Default: highest configured level when unknown.
- **Tool outputs.** Results from previous actions.
- **Semantic distance.** Drift from original request (see Section 8.4).
- **Identity context.** Verified identities of agent, user, and tools.

8.4 Semantic Distance Tracking

The runtime SHOULD compute semantic distance between actions and stated intent:

$$d(r_0, a_n) = 1 - \cos(\text{embed}(r_0), \text{embed}(a_n))$$

where r_0 is the original request and a_n is the current action. Cumulative drift SHOULD be tracked across sequences. Thresholds are deployment-specific and SHOULD be calibrated empirically.

Semantic distance is a *risk signal*, not a primary authorization primitive. The hard authorization layer in OATS is the deterministic policy engine (Cedar, OPA, or equivalent) evaluating structured inputs. Semantic distance provides an advisory signal that the policy engine may consume as one input to step-up or defer decisions, but OATS-compliant runtimes SHOULD NOT rely on semantic distance as the sole basis for irreversible denial. This separation ensures that the Gate’s authorization decisions remain deterministic and reproducible even when the drift signal is heuristic. The limitations of embedding-based distance tracking are discussed in Section 15.

8.5 Step-Up Authorization and Deferral

For step-up authorization: execution MUST block until approval; full context MUST be available to approvers; configurable timeouts MUST be enforced (deny on timeout); decisions MUST be recorded in the journal.

For deferral: deferred actions MUST remain paused without effects; the runtime MUST track deferred actions and maintain execution order; cascading deferrals MUST be bounded (deny when limit exceeded); deny on timeout MUST be the default; both deferral and resolution MUST be recorded.

8.6 Defense in Depth at the Action Layer: Two Independent Fences

Empirical results from stack-stripping ablation experiments on the reference implementation [27, §6.8][25] establish a previously under-emphasized architectural property: the action layer in an

OATS-compliant runtime is two independent fences operating in series, not one. An OATS-compliant runtime SHOULD make this distinction explicit and verify both fences independently.

Fence A: the policy engine. Cedar (or equivalent) evaluates the tuple (a, C, I) against organizational policy and returns an authorization decision. This is the fence specified in Section 8.1 through Section 8.5. It reasons over typed action proposals and accumulated session context.

Fence B: the executor profile-of-one. Each principal’s tool executor (`ActionExecutor` or its equivalent) MUST maintain a static handler-map declaring exactly the set of tool names the principal is permitted to dispatch. Before dispatching any approved action, the executor MUST verify that the action’s tool name is a member of the principal’s declared profile. Tool names not in the profile MUST be refused even if the policy engine permits them. This refusal is independent of policy evaluation: it is a name-membership check against a static set.

Why two fences. The two fences operate at different abstraction levels and fail in different ways. Cedar evaluates policies whose correctness depends on the policy author anticipating attack patterns; the executor profile-of-one evaluates against a static name list that is enumerable, reviewable, and amenable to formal analysis. Empirical ablation [27, §6.8] showed that with Cedar disabled in a 434-row tool-confusion sweep, the executor profile-of-one fence still refused 219 out-of-profile dispatches — a result the runtime would not have achieved with a single-fence action layer. The independence of the two fences is what produces this defense-in-depth property.

Conformance implication. Conformance requirement C3 (Section 12) is satisfied by implementing either fence; conformance to the defense-in-depth property requires both. An OATS Extended-conformant implementation SHOULD implement both fences and document the independence guarantee. The two fences are particularly valuable in deployments where the policy engine is dynamically configurable (and thus may be misconfigured) while the principal tool profiles are statically declared at compile time or deployment time.

9 Audit Layer

9.1 Journal Architecture

An OATS-compliant runtime MUST maintain a cryptographic audit journal. The journal is the authoritative record of what happened, when, why, and by whose authority.

```
Entry  $n$ : { event, ts, prev_hash, payload, signature }
      prev_hash = SHA-256(canonical(Entry  $n-1$ ))
      signature = Ed25519(canonical(Entry  $n$ ))
```

Figure 6: Hash-chained audit journal. Each entry includes the cryptographic hash of the previous entry and an Ed25519 signature, forming a tamper-evident chain verifiable offline.

Event	When	Content
LoopStarted	Loop begins	Configuration, agent identity, original request
ReasoningComplete	After LLM, before Gate	Proposed actions, token usage
PolicyEvaluated	After Gate decision	Decisions, matching policies, reasons
ToolsDispatched	After execution	Tools, parameters, duration, evidence hashes
ObservationsCollected	After results	Observation count, context size
LoopTerminated	Loop ends	Reason, iterations, total usage, duration
RecoveryTriggered	On failure	Strategy, error context

Table 6: Required journal event types.

9.2 Event Types

9.3 Cryptographic Properties

Each entry **MUST** include an Ed25519 signature (or ECDSA P-256) covering the canonical serialization, a hash chain link to the previous entry, and a cryptographic timestamp. Entries **MUST** be verifiable offline.

9.4 Evidence Envelopes

Tool executions **MUST** produce structured evidence envelopes: tool name and version, validated parameters, constructed invocation, duration and exit status, output hash (SHA-256), authorizing policy decision, and identity at execution time.

9.5 Compliance Properties

The journal provides infrastructure that can contribute to regulatory compliance, though OATS alone is not sufficient for any regulatory framework. Specifically: the journal can serve as a component of HIPAA audit trails (recording health data access with identity and authorization); SOC2 evidence collection (recording policy enforcement decisions); SOX audit trail requirements (recording attributable financial system actions); and GDPR accountability mechanisms (recording data access patterns). In each case, the journal addresses the technical recording requirement but does not address the organizational, procedural, or legal requirements of the applicable regulation.

9.6 Redaction of Sensitive Parameters

Journal entries describe tool dispatches, including their validated parameters. Some of those parameters carry secrets (API keys, tokens, passwords) whose presence in a long-lived audit log creates risk disproportionate to their auditing value. Tool contracts **SHOULD** declare which parameters are sensitive (via a `sensitive_params` annotation or equivalent), and journal writers **SHOULD** substitute a fixed redaction sentinel (e.g., "[REDACTED]") for those parameters' values when writing entries. The parameter *names* and the *fact of dispatch* **MUST** still be recorded; only the values are redacted. Evidence-envelope output hashes remain over the original parameters so verification still works for non-secret parameter fields.

10 Sandboxing and Isolation

10.1 Multi-Tier Sandboxing

An OATS-compliant runtime SHOULD support multiple sandboxing tiers:

Tier	Mechanism	Isolation level	Overhead
1	Container (Docker)	Process, filesystem, network	Low
2	User-space kernel (gVisor)	Syscall filtering	Medium
3	MicroVM (Firecracker)	Hardware-level	Medium–High

Table 7: Sandboxing tiers.

10.2 Resource Limits

Agent execution MUST support configurable limits: token budget, time budget, iteration budget, tool call budget, network restrictions, and filesystem restrictions.

10.3 Circuit Breakers

Tool executions SHOULD be protected by circuit breakers. When a tool fails repeatedly, the circuit trips and subsequent calls are rejected until reset, preventing cascading failures and runaway retry loops.

11 Inter-Agent Communication

11.1 Communication Governance

All inter-agent messages MUST pass through a communication policy gate evaluating authorization rules on communication primitives (`ask`, `delegate`, `send`, `parallel`, `race`).

11.2 Message Security

Messages MUST be cryptographically signed (Ed25519), encrypted (AES-256-GCM), and attributed to verified agent identities.

11.3 Delegation Constraints

Delegation chains MUST be bounded: maximum depth (configurable), capability narrowing (delegated agents cannot exceed delegating agent’s capabilities), and blast-radius containment.

11.4 Cross-Agent Context

Session context (original request, prior actions, data classifications) SHOULD be propagated to downstream agents, enabling their Gates to evaluate against original intent.

11.5 Distributed Trace Context

New in v1.3.0.

Multi-agent workflows that span runtime boundaries are difficult to investigate when journal entries from each runtime cannot be stitched into a single causal chain. To enable forensic reconstruction across runtimes, OATS-compliant runtimes **SHOULD** propagate W3C Trace Context [20] across agent boundaries:

- Outbound inter-agent messages **SHOULD** include a **traceparent** header carrying the trace ID and parent span ID.
- Inbound messages **SHOULD** extract **traceparent** and bind the resulting span to the receiving agent’s journal entries.
- Cron-scheduled and heartbeat-triggered agent invocations **SHOULD** originate a new trace and record the trace ID in their **LoopStarted** event.

Trace context propagation **MUST NOT** be a load-bearing security primitive; it serves observability and forensic reconstruction, not authorization. The hard cryptographic identity binding remains Section 11.2 and Section 7.

12 Conformance Requirements

The requirement language follows RFC 2119: **MUST** indicates absolute requirements; **SHOULD** indicates recommendations that may be omitted with documented justification.

12.1 Conformance Levels

OATS Core (all **MUST** requirements, C1–C7): Baseline zero-trust agent execution.

OATS Extended (all **MUST** and **SHOULD** requirements, C1–C7 + E1–E9): Comprehensive zero-trust with identity, sandboxing, content sanitization, and advanced policy.

12.2 Core Requirements (**MUST**)

C1: ORGA Loop Enforcement. The runtime **MUST** implement the four-phase ORGA loop. The Gate **MUST** execute before every tool dispatch. In compiled languages, phase transitions **MUST** be enforced at compile time via typestates. In interpreted languages, equivalent enforcement **MUST** be provided and documented, with acknowledgment of residual risk per Section 5.5.

Verification. Attempt to construct a code path from Reason to Act bypassing Gate. In a typestate implementation, this **MUST** be a compile error. In runtime-enforced implementations, this **MUST** be caught by a verified test suite with 100% tool dispatch path coverage.

Empirical evidence. Demonstrated in the reference implementation through nine compile-fail tests under **tests/compile-fail/** [27, §6.9][25]. Each test exhibits the expected compiler diagnostic for an illegal state transition or feature-gating violation; the suite re-verifies on every CI run.

C2: Tool Contract Support. The runtime **MUST** support declarative tool contracts with typed parameter validation. The LLM **MUST NOT** generate raw tool invocations. All invocations **MUST** be constructed from validated parameters and contract-defined templates.

Verification. Submit parameters containing shell metacharacters (;, |, &, '). Verify rejection. Submit parameters outside declared type constraints. Verify rejection. Verify the LLM never receives raw invocation strings in any code path.

Empirical evidence. Demonstrated in the reference implementation through the v11 ToolClad sweep [23, §5]: 333 of 335 treatment-arm dispatches refused across nine widely available hosted LLMs and eight attack sub-shapes (metacharacter, command substitution, backtick, wildcard, newline, path traversal, Cyrillic-homoglyph IDN, punycode IDN). The two non-refusals were clean-hostname self-sanitisation cases; counted strictly against hostile inputs the refusal rate was 100%. The substrate-comparison sweep [24, §5] reproduces this property at 0/560 escape on four pure-action vectors against Symbiont, vs. 88–98% baseline rates on `python_baseline` and `python_sandboxed` substrates with the same lures and models.

C3: Policy Evaluation. The runtime MUST evaluate actions against policy before execution. The policy engine MUST operate outside LLM influence. MUST support Allow, Deny, Modify, Step-Up, and Defer decisions. Default MUST be deny.

Verification. Configure a DENY policy. Submit a matching action. Verify no effects on the target system. Verify denial recorded in journal with matching policy and reason. Repeat for each decision type.

Empirical evidence. Demonstrated in the reference implementation through the v9 cloud-adversarial sweep [27, §6.2]: 263 forbidden tool-call attempts refused across 874 cloud-adversarial runs at \$9.88 of authoritative billing, with zero attempts reaching execution. Cumulative across all sweeps: 642 Cedar plus 34 executor refusals across nine models. The two-fences-at-action-layer property described in Section 8.6 was independently validated through stack-stripping ablation [27, §6.8].

C4: Context Accumulation. The runtime MUST accumulate session context across actions. Context MUST include original request (when available), action history, and data classification.

Verification. Execute a sequence of three or more actions. Verify the policy engine receives accumulated context for each subsequent action. Verify context includes prior actions and their data classifications.

Empirical evidence. Operationally demonstrated in the reference implementation; rigorous evaluation across context lengths is identified as future work.

C5: Cryptographic Audit Journal. The runtime MUST maintain a hash-chained, cryptographically signed audit journal recording all ORGA loop events (Table 6). Entries MUST be verifiable offline.

Verification. Generate journal entries for allowed, denied, deferred, and step-up actions. Verify all fields present, signatures valid, and hash chain intact. Tamper with one entry and verify that chain verification detects the modification.

Empirical evidence. Operationally demonstrated through per-call audit sidecars produced by every sweep in the reference corpus [25]; the per-call JSONL format records every ORGA loop event and is the artifact from which the empirical numbers in [27], [23], and [24] are reproduced. Targeted tamper-resistance evaluation is identified as future work.

C6: Gate Independence. The Gate MUST operate on structured inputs only. It MUST NOT process natural language, share mutable state with the LLM, or be influenced by the LLM’s reasoning.

Verification. Inspect Gate implementation. Verify inputs are typed structs (tool name, operation, parameters, identity, context), not natural language strings. Verify no shared mutable references between the Gate and the LLM inference component. Verify no dynamic code paths within the Gate that are parameterized by LLM output.

Empirical evidence. Architecturally demonstrated through the typestate construction in the reference implementation [27]; targeted Gate-influence probing is identified as future work.

C7: Evidence Envelopes. Tool executions MUST produce structured evidence envelopes with output hashes, execution metadata, and identity binding.

Verification. Execute a tool. Verify the envelope contains tool name, version, validated parameters, constructed invocation, duration, exit status, SHA-256 output hash, authorizing policy decision, and agent/user identity.

Empirical evidence. Operationally demonstrated through the per-call JSONL audit sidecars produced by every sweep in the reference corpus [25]. Each tool dispatch produces an envelope conforming to the C7 schema.

12.3 Extended Requirements (SHOULD)

E1: Tool Integrity Verification. Verify tool contract signatures using domain-anchored cryptographic verification with TOFU key pinning. Verifiers SHOULD enforce a per-credential-class algorithm allowlist (§7.5) that rejects unlisted algorithms at both the header-inspection guard and the verification API.

E2: Agent Identity Verification. Verify agent identity using domain-anchored ES256 credentials with delegation chain support. JWT verifiers SHOULD reject `none`, `RS*`, `PS*`, and `HS*` algorithms on asymmetric paths (§7.5) and SHOULD validate `aud`, `iss`, and `exp` claims unconditionally.

E3: Semantic Distance Tracking. Compute and track semantic distance between actions and stated intent using embedding similarity.

E4: Multi-Tier Sandboxing. Support configurable sandboxing tiers (container, kernel-level, microVM).

E5: Inter-Agent Communication Governance. Enforce authorization policies on inter-agent communication with signed and encrypted messages. SHOULD propagate W3C Trace Context (§11.5) for cross-runtime forensic reconstruction.

E6: Telemetry Export. Export structured telemetry (OCSF, CEF, or documented custom schemas) with real-time streaming.

E7: Formally Verifiable Policies. Use a policy language enabling static analysis of correctness (Cedar, OPA, or equivalent).

E8: Least-Privilege Credential Scoping. Support just-in-time credential issuance with operation-specific scoping and logged usage.

E9: Content Sanitization (new in v1.3.0). Sanitize agent-influenced string fields before they reach reasoning context, the journal, or downstream tools per §6.6. At minimum SHOULD remove ASCII C0/DEL, C1, zero-width, bidi-override, word-joiner, BOM, variation-selector, Unicode Tag block, soft hyphen, and combining-diacritical characters; SHOULD apply NFKC normalization and flag unusual script mixing.

Verification. Submit each of the following payloads as a string parameter and verify the sanitized output, the journal entry, or both contain no instance of the hostile character class: (a) U+200B-padded content; (b) U+202E bidi-override prefix; (c) U+E0000 Unicode-tag-encoded content; (d) NFKC-decomposable fullwidth Latin letters; (e) Latin+Cyrillic mixed-script identifier with at least one Cyrillic confusable.

13 Implementation Architectures

OATS does not mandate a specific implementation architecture. The specification intentionally separates conformance properties from implementation details and includes four deployment patterns to reduce dependence on any single implementation. The current specification is informed by one reference implementation (Symbiont [26]); independent conformance testing across additional implementations is needed to validate that the specification is sufficiently general. Comparative substrate evidence [24] establishes that the structural-enforcement properties produce measurable defense beyond OS-isolation baselines, but does not substitute for independent OATS-compliant runtime implementations. Table 8 compares the four reference architectures.

Property	Self-Hosted	Plugin/Ext.	Gateway	Vendor Integ.
You control	Everything	Agent code	Network	Policy only
Enforcement	ORGA typestate	Dual-layer	Network proxy	Vendor hooks
Bypass resistance	Very high	High	High	Vendor-dependent
Context richness	Full	Full (inner)	Limited	Vendor-dependent
Tool contracts	Full	Full (outer)	Partial	Vendor-dependent
Identity	Full	Full (outer)	Partial	Vendor-dependent
OATS-conformant	Yes	Yes	Partial	If hooks sufficient

Table 8: Implementation architecture comparison.

13.1 Self-Hosted Runtime

The full OATS stack deployed as a single runtime. Provides the strongest available enforcement properties: compile-time phase enforcement, full context visibility, cryptographic identity, and multi-tier isolation. Natural home in systems-level languages with rich type systems.

13.2 Plugin/Extension Model

For agents running inside third-party platforms. An inner layer (plugin) provides awareness; an outer layer (OATS runtime wrapping the platform via CLI executor or container) provides enforcement. Because the outer ORGA Gate mediates all tool invocations at the process boundary, the inner platform cannot bypass it through normal operation, though side-channel bypasses (e.g., direct network calls from within the sandbox) require complementary network-level controls.

13.3 Gateway Architecture

For protocol-based tool invocations (MCP, REST). An OATS-compliant gateway intercepts traffic between agents and tools, implementing Gate, context accumulation, and journaling. Provides enforcement without agent modification, at the cost of reduced context visibility.

13.4 Vendor Integration

For SaaS agents where organizations control no infrastructure. Requires vendor-provided synchronous pre-execution hooks, decision enforcement, context availability, and receipt export. OATS provides the specification for vendor evaluation and contracts.

14 Evaluation Framework

The claims made in this specification are architectural: OATS is designed to provide certain security properties through structural enforcement. Converting these design-level claims into empirical evidence requires a systematic evaluation methodology. Section 14.1 through Section 14.6 define the framework; Section 14.7 reports initial empirical results from the reference implementation that validate five of seven core conformance requirements, including comparative substrate evidence newly available in v1.3.0.

14.1 Attack Suite Methodology

To measure whether OATS reduces attack success rates, we define a comparative evaluation against three baselines:

Configuration	Description
Baseline A	No policy enforcement. Agent invokes tools directly.
Baseline B	Deny-list policy. Agent actions intercepted and evaluated against forbidden-action rules.
Baseline C	Prompt-guardrail only. Input/output filtering at the LLM layer, no action-level enforcement.
OATS	Full stack: ORGA loop, tool contracts, policy engine, identity verification, journal.

Table 9: Baseline configurations for the attack suite.

The attack suite combines existing benchmarks with custom scenarios:

- **AgentDojo [4]**. Dynamic environment evaluating prompt injection attacks and defenses across realistic agent tasks. We measure attack success rate (percentage of injections that produce the attacker’s intended tool invocation) and task utility (percentage of legitimate tasks completed successfully).
- **Custom injection suite**. 200+ prompt injection variants (direct, indirect via documents, indirect via tool outputs, multi-turn) targeting tool invocations across 10 tool types. Includes attacks specifically targeting allow-list bypass (e.g., attempting to construct shell commands through parameter concatenation).
- **Compositional exfiltration scenarios**. 50 multi-step sequences where individual actions are policy-compliant but the composition constitutes a violation (e.g., read sensitive data then email externally).

For each configuration, we report: attack success rate, task completion rate (legitimate tasks), false positive rate (legitimate actions blocked), and false negative rate (attack actions allowed).

Fairness methodology. To prevent evaluation bias toward the OATS architecture, the following constraints apply: (a) all configurations use the same task suite, tool set, and underlying LLM; (b) credential scopes are identical across configurations; (c) deny-list policies in Baseline B are tuned using a held-out calibration set, not the test set; (d) all policy thresholds and drift thresholds are fixed before test execution and not adjusted post-hoc; (e) all failures, false negatives, and bypass successes are reported, not only aggregate metrics.

14.2 Performance Overhead

Runtime enforcement introduces latency. We define benchmarks for each enforcement layer:

Component	Measurement	Target
Static policy evaluation	Per-action latency (p50, p95, p99)	< 1 ms
Context-dependent evaluation	Per-action latency with accumulated context	< 5 ms
Semantic distance computation	Per-action embedding + cosine similarity	< 50 ms
Tool contract validation	Parameter type checking + sanitization	< 0.5 ms
Ed25519 journal signing	Per-entry signature generation	< 0.1 ms
Hash chain verification	Per-entry hash computation + chain link	< 0.05 ms
Full ORGA Gate pipeline	End-to-end from action proposal to allow/deny	< 10 ms (p99)
Evidence envelope generation	SHA-256 output hash + envelope construction	< 1 ms

Table 10: Performance benchmarks for OATS enforcement components. Targets are derived from operational experience and represent design goals; measured values from the reference implementation are reported in Section 14.7.

14.3 False Positive/Negative Analysis

Security enforcement is useful only if it blocks attacks without blocking legitimate work. We define a methodology for measuring this tradeoff:

- **Task suite.** Run BFCL (Berkeley Function Calling Leaderboard) and SWE-bench Lite task sets through an OATS-compliant runtime.
- **False positive rate.** Percentage of legitimate tool invocations that the Gate denies or defers. Measured across policy strictness levels.
- **False negative rate.** Percentage of attack-injected tool invocations that the Gate allows. Measured using the attack suite from Section 14.1.
- **ROC analysis.** Plot false positive rate vs. true positive rate (attack blocked) across policy threshold configurations.

14.4 Bypass Resistance Testing

Each enforcement mechanism has a theoretical bypass boundary. We define tests that probe these boundaries:

- **Typestate circumvention.** Attempt to construct Rust code paths from Reasoning to ToolDispatching that skip PolicyCheck. The compiler should reject all such paths.
- **Tool contract metacharacter injection.** Submit parameters containing shell metacharacters, SQL injection patterns, path traversal sequences, and Unicode homoglyphs to each parameter type.
- **Gate influence probing.** Craft LLM outputs designed to influence Gate evaluation.
- **Sandbox escape.** Attempt network calls, filesystem access, and process spawning from within each sandbox tier.
- **Identity spoofing.** Attempt to present forged AgentPin credentials, tampered SchemaPin signatures, and replayed JWTs.
- **Algorithm-confusion attacks.** Attempt to present a JWT signed under an algorithm not on the verifier’s allowlist (e.g., `none`, RS256 against an ES256-only verifier) and confirm refusal both at header inspection and at the algorithms field of the verification call.
- **Substrate comparison.** Run identical lures against permissive, OS-isolated, and structurally-enforced substrates with the same models, harness, and trial count to disentangle structural-enforcement contribution from OS-isolation contribution [24, §3].

14.5 Ablation Study

To measure the marginal contribution of each OATS layer, we define an ablation removing one layer at a time:

Configuration	Layers active	Expected impact
Full OATS	All 5 layers	Baseline (best security, highest overhead)
No contracts	ORGA + policy + identity + journal	Allows arbitrary action formulation
No identity	ORGA + contracts + policy + journal	Removes mutual auth
No context	ORGA + contracts + static policy + journal	Removes context-dependent classifications
No journal	ORGA + contracts + policy + identity	Removes audit trail
No sanitization	All 5 layers, sanitizer disabled	Exposes content-shape vectors
ORGA only	Loop enforcement, permissive policy	Tests phase ordering alone

Table 11: Ablation study configurations.

14.6 Case Studies

Three detailed scenarios that exercise multiple OATS layers simultaneously: (1) multi-step data exfiltration with prompt-injected document; (2) tool supply chain attack via tampered contract; (3) intent drift across a long session. Detailed scenario specifications are available in [25].

14.7 Initial Empirical Results

Initial empirical results from the reference implementation are now available through the `symbiont-orga-demo` reference corpus [25] and three companion preprints [27, 23, 24]. This section summarizes the headline findings; full methodology, per-model breakdowns, and reproduction artifacts are reported in the cited works.

Methodology summary. All sweeps run against nine widely available hosted LLMs in April–May 2026 (GPT-5, Claude Haiku 4.5, Gemini 2.5 Pro, DeepSeek-V3.1, Qwen3-235B, Qwen3.6-Plus, MiMo-V2-Pro, MiniMax-M2.7, gpt-oss-20b) routed through OpenRouter. The substrate-comparison sweep additionally tests Claude Sonnet 4.6, DeepSeek v4 Pro/Flash, Qwen 3.6 Max/27B and substitutes Qwen 3.5 35B for Qwen 3.6 35B. Each sweep produces per-call JSONL audit sidecars from which the reported numbers are reproduced. Total authoritative billing across the corpus is approximately \$80 over the v9 through v12 sweeps plus the substrate-comparison sweep [24, App. C] reports \$50.89 OpenRouter spend for the substrate sweep alone; reproduction scripts and the frozen sidecar databases are published with the cited preprints.

Attack-suite results (validates C3, partial validation of C2). The v9 cloud-adversarial sweep [27, §6.2] tested ten attack shapes against the policy gate across 874 cloud-adversarial runs. Result: 263 forbidden tool-call attempts refused, zero attempts reaching execution. Cumulative across all sweeps: 642 Cedar denials plus 34 executor refusals. The false-positive baseline across 813 legitimate runs on eight of nine models was zero Cedar denials; the gpt-oss-20b case (82 denials on legitimate work) was explained as a model misconception correctly caught rather than a false positive.

Tool-contract results (validates C2). The v11 ToolClad sweep [23, §5] tested eight argument-injection sub-shapes (metacharacter, command substitution, backtick, wildcard, newline, path traversal, Cyrillic-homoglyph IDN, punycode IDN) against a `whois_lookup` tool with a `scope_target` parameter type. Result: 333 of 335 treatment-arm dispatches refused across nine models. The two non-refusals were clean-hostname self-sanitisation cases (the model refused the injection prompt and emitted `example.com` instead of the canary payload); counted strictly against hostile inputs the bite-rate was 100%. The v0.1 of [23] reported a published limitation (the as-built `scope_target` validator accepted punycode hostnames as a side effect of its ASCII-only rule); ToolClad v0.6.0 closed that gap with explicit punycode-label rejection, and the v0.2 sweep added a punycode-idn sub-shape that empirically validated the fix at 33 of 33 refusals.

Substrate-comparison results (validates the architectural-contribution claim). *New in v1.3.0.* The substrate-comparison sweep [24] runs identical lures across three substrates — `python_baseline` (no enforcement), `python_sandboxed` (Docker isolation), and `Symbiont` (full OATS stack) — with the same harness, models, and sample sizes. Across four pure-action vectors (`fs_enumerate`, `fs_traversal`, `network_egress`, `dispatch_confusion`), pooled escape rates were:

Vector	<code>python_baseline</code>	<code>python_sandboxed</code>	<code>Symbiont</code>
<code>fs_boundary</code> (control)	0% (0/140)	0% (0/140)	0% (0/140)
<code>fs_enumerate</code>	88% (123/140)	88% (123/140)	0% (0/140)
Pooled across 4 pure-action vectors	88–98%	88–98%	0% (0/560)
orchestrator-injection (n=200)	59%	56%	3.6%

Table 12: Substrate-comparison escape rates across four pure-action vectors and the orchestrator-injection content-shape vector.

This produces the substrate-comparison story: container isolation provides effectively zero defense against in-bind-set scope expansion, while structural-shape enforcement eliminates escape on the same vectors regardless of model capability tier. The 88–98 percentage-point reduction

substantiates that the OATS-specified properties produce measurable defense beyond OS-isolation alternatives — the comparative claim that single-substrate sweeps cannot make.

Content-shape ceiling (the GPT-5 outlier). *New in v1.3.0.* The orchestrator-injection vector measures content-shape attacks that pass shell-metacharacter validation but carry adversarial semantic content (e.g., HTML-comment-smuggled instructions in tool result strings). On this vector at $n = 200$ with Wilson 95% confidence intervals, six of seven evaluated models cluster at 1–4% bypass against Symbiont’s combined defenses; GPT-5 alone reaches 16% (12%, 22%), a confidence interval that does not overlap any other model’s CI. The substrate paper characterizes this finding as the *regex ceiling*: substring-based content fences are necessary but structurally bounded — every marker-list expansion catches the previous bypass class; sufficiently capable models invent the next class. This finding refines (rather than contradicts) the architectural claim: action-shape vectors are structurally defended at 0%, while content-shape vectors require additional structural moves at the validator or at the orchestrator’s prompt construction to close further. See Section 15 for the formal limitation.

Ablation results: fence non-redundancy (validates the layered-defense claim). The v12 stack-stripping ablation [27, §6.8] disabled each fence layer in turn and measured the corresponding attack-class survival rate. Removing the policy gate exposed 100% of out-of-profile dispatches (115 v5-baseline denials drop to zero with Cedar disabled). Removing the content sanitiser exposed 92.2% of html-comment-smuggle payloads (389 of 422 stored procedures contained intact hostile content with the sanitiser disabled). Removing ToolClad exposed 99.4% of tool-arg-injection payloads on the v11 corpus. Across three attack classes and three fence types, each fence is empirically necessary to defend its class.

Two-fences-at-action-layer finding (validates §8.6). The Cedar ablation produced an unanticipated result: with Cedar disabled, the executor profile-of-one fence still refused 219 out-of-profile dispatches across 434 rows. This demonstrated that the action layer is two independent fences operating in series, not one. Cedar evaluates policies whose correctness depends on the policy author anticipating attack patterns; the executor profile-of-one evaluates against a static name list. The two fences are independent; removing one does not remove the other. This finding is now formalized in Section 8.6 as a SHOULD requirement for OATS Extended conformance.

Performance results (validates Table 10 targets). Instrumented measurements from the v10 sweep [27, §6.4]: the Cedar policy gate adds 30–95 μ s per call (well below the 10 ms p99 target in Table 10). The content sanitiser adds approximately 345 ns per call (well below the 0.5 ms target). End-to-end agent-loop latency is dominated by LLM inference round-trip by four to seven orders of magnitude over any plausible runtime overhead from these enforcement components. Performance is empirically not a barrier to adopting the structural-enforcement model.

Compile-fail tests (validates C1). Nine compile-fail tests in the reference implementation [27, §6.9] verify the tpestate property on every CI run by exhibiting the expected compiler diagnostic for each illegal state transition. The tests cover skipping the policy check, observing without dispatching, retaining handles after move, constructing tpestate values externally, accessing phase-specific data from the wrong phase, and feature-gating violations. The suite re-verifies on every commit; the tpestate property is therefore not just specified but build-time enforced.

Reference implementation hardening milestone. *New in v1.3.0.* The Symbiont v1.14.0 release (May 2026) [26] responded to an independent security audit covering 5 critical, 7 high, 10 medium, and 9 low findings. Of relevance to OATS:

- **Fail-closed construction.** `DefaultPolicyGate::new()` now returns `Deny` for every `ToolCall` and `Delegate` action with an explicit reason; the prior hard-coded `permissive()` default was removed. This aligns the operational default with the spec-level default-deny in §8.1 and motivated the explicit fail-closed-construction language added there in v1.3.0.
- **JWT algorithm allowlist.** ES256 / EdDSA enforced for asymmetric Bearer; HS256 enforced for HMAC webhook signatures; RS* / PS* / `none` refused at both the header guard and the validator. This neutralizes RUSTSEC-2023-0071 on every operator-controlled path and motivated §7.5.
- **JSON Schema argument validation pre-Gate.** Tool-call arguments are validated against the declared JSON Schema before the policy gate evaluates; non-object arguments and schema-violating arguments produce `Deny`. Reinforces the allow-list ordering specified in §8.2 (Figure 5).
- **symbi-invis-strip v0.3.0.** Forbidden range expanded to include U+00AD, U+0300–036F (combining diacriticals), and U+2070–209F (super/subscript forms); `detect_injection_patterns` now NFKC-normalizes and adds a compact-projection scan plus mixed-script (Latin+Cyrillic) flagging. Motivated the §6.6 SHOULD language.

These changes are implementation milestones, not changes to the specification. They demonstrate that the v1.3.0 SHOULDs are tractable in a production-grade reference implementation.

What is not yet measured. The reference corpus does not yet provide direct empirical evidence for: (C4) context accumulation across long sessions; (C6) Gate independence under targeted influence probing; (C5) audit journal tamper-resistance under adversarial conditions; (E1, E2) AgentPin and SchemaPin verification protocols under adversarial probing including algorithm-confusion; (E4) sandbox escape testing across the three sandboxing tiers; (E9) content-sanitization conformance under expanded adversarial corpora; case studies as specified in Section 14.6; cross-language conformance across the Python, JavaScript, and Go reference implementations of ToolClad. These are identified as future deliverables. Multi-implementation conformance (an independent OATS-compliant runtime) remains the most important next deliverable.

15 Limitations

This section identifies known limitations of the OATS specification. These are not future research directions (Section 16) but inherent boundaries of the current architecture.

Typestate scope. Compile-time enforcement of the ORGA loop applies only to code paths within the typestate-governed loop. Agent code that bypasses the loop entirely — for example, by invoking tools through a separate code path not mediated by the ORGA runner — is not caught by the type system. Sandboxing and network isolation provide complementary enforcement but are defense-in-depth layers, not compile-time properties.

Tool contract coverage. The allow-list model governs only tools with declared contracts. Tools without contracts (legacy integrations, dynamically discovered MCP servers, ad-hoc API calls) are outside the allow-list boundary. An OATS-compliant runtime can deny uncontracted tool invocations by default, but this trades functionality for safety and may be impractical in environments with large numbers of tools.

Coverage-safety tradeoff. The allow-list model inherently restricts the agent’s action space. Novel legitimate tool uses that were not anticipated when the contract was authored will be rejected until the contract is updated. This creates operational friction proportional to the rate of tool evolution. The severity of this tradeoff has not been quantified empirically.

Content-shape ceiling on capable models. *New in v1.3.0.* String-based content sanitization (§6.6) defends a defined character set: invisible Unicode, bidi overrides, fullwidth homoglyphs, and similar. It does not defend semantic content-shape attacks that compose legitimate characters into adversarial meaning. Empirical evidence [24, §7] shows that against the most capable evaluated model (GPT-5), the orchestrator-injection vector retains ~16% bypass at $n = 200$ even with the full marker-list content sanitizer enabled. Six of seven other evaluated models cluster at 1–4%. The substrate paper formalizes this as the *regex ceiling*: every marker-list expansion catches the previous bypass class; sufficiently capable models invent the next class. Closing this gap requires structural moves at the validator (typed agent-summary content with grammar-bound output schemas) or at the orchestrator’s prompt construction, not further marker expansion. The architectural claim that action-shape vectors are structurally defended at 0% across capability tiers is not affected by this finding.

Answer-correctness orthogonal to structural fences. *New in v1.3.0.* Adversarial tool-result content can degrade an agent’s final answer in ways that cross no policy fence, no content sanitizer, and no structural validator [27, §6.7]. The typestate preprint reports a case in which a tool returned a misleading-but-syntactically-legitimate diagnostic string that caused the agent to commit a wrong-but-plausible answer; the task grader, not the structural fences, caught the harm. OATS specifies *runtime action security*. Answer-correctness verification (graders, output-shape validators against task-specific expected answers, downstream consensus or critic agents) is a complementary fence that OATS does not currently cover. Deployments that care about answer correctness in adversarial environments SHOULD treat the grader as a load-bearing fence, not a footnote.

Semantic distance limitations. Drift detection via embedding similarity depends on the quality of the embedding model and the meaningfulness of cosine distance in the action-intent space. Adversarial embeddings could subvert drift detection by producing actions that are semantically distant from the original intent but close in embedding space. The robustness of semantic distance tracking under adversarial conditions has not been evaluated.

Single reference implementation. *Updated in v1.3.0.* The specification is informed by one reference implementation (Symbiont [26]). Multi-implementation conformance testing — building independent OATS-compliant runtimes and verifying interoperability — has not been conducted. The substrate-comparison sweep [24] establishes that the OATS-specified properties produce measurable defense versus alternative substrates (permissive Python, container-isolated Python), but tests only one OATS-compliant implementation against those non-OATS alternatives; the question of whether independently-built OATS-compliant runtimes converge on the specified behavior remains open.

The specification may contain implicit assumptions derived from the reference implementation that create unnecessary barriers for alternative implementations.

Production case study. The empirical results in Section 14.7 are from benchmark workloads run against hosted LLM endpoints, not from production deployment telemetry. While the reference implementation has operated autonomously for approximately nine months, no controlled production case study has been published. Adopting OATS in environments with regulatory or audit requirements should include independent operational evaluation.

Regulatory insufficiency. The audit journal provides technical infrastructure for compliance but is not sufficient for any regulatory framework on its own. HIPAA, SOC2, SOX, and GDPR each impose organizational, procedural, and legal requirements that OATS does not address. Claiming OATS compliance should not be conflated with claiming regulatory compliance.

Deferral latency. The DEFER authorization decision suspends action execution until resolution. In time-critical agent workflows (e.g., real-time trading, incident response), deferral latency may be unacceptable. The specification does not provide guidance on latency-sensitive deferral policies beyond configurable timeouts.

Privacy in cross-agent context. Propagating session context across agent boundaries (§11.4) raises privacy and data sovereignty concerns. Context may contain sensitive information from the original user’s request, and propagating it to downstream agents in different organizational domains may violate data handling agreements. The specification does not address context redaction or privacy-preserving context propagation.

Non-deterministic evaluation. Context-dependent action classification relies on the policy engine’s evaluation of accumulated context. When the policy engine uses semantic similarity or ML-based classification, evaluation results may be non-deterministic across invocations. The specification requires deterministic policy engines (Cedar, OPA) but permits semantic distance as a SHOULD requirement, creating a tension between deterministic authorization and non-deterministic drift signals.

16 Research Directions

16.1 Typestate in Non-Rust Languages

OATS’s compile-time enforcement property is most naturally expressed in languages with typestate support (Rust, Haskell, Scala). Providing equivalent enforcement in Python, JavaScript, and Go requires runtime checks with formal path coverage verification, or code generation from a verified specification. The degree of assurance loss when moving from compile-time to runtime enforcement is an open question.

16.2 Data Flow Through Context Windows

Data may be transformed, summarized, or paraphrased by the LLM before use in subsequent actions. Information-theoretic approaches (taint analysis, embedding watermarking) for tracking lineage through non-deterministic transformations are active research.

16.3 Multi-Agent Trust Coordination

Maintaining coherent trust chains across organizational boundaries in delegation requires distributed tracing standards, federated receipt verification, and cross-domain policy negotiation.

16.4 Formal Verification of the ORGA Loop

Typestate enforcement addresses phase ordering within the loop. Mechanized proofs of the entire system — policy engine correctness, context accumulator completeness, journal integrity, and the absence of bypass paths outside the loop — would provide substantially stronger assurance.

16.5 Approval Fatigue and Deferral Resolution

Balancing security against usability. ML-based approval recommendation, batch approval, and progressive autonomy (reduced approval requirements through demonstrated compliance) are active directions.

16.6 Vector Embedding Security

When semantic distance tracking uses embeddings, those embeddings become a security surface. Information-theoretic watermarking, steganographic attack detection, and quantization-robust integrity verification are needed. Early work along this line includes *VectorSmuggle* [28], which empirically characterises steganographic exfiltration through vector-store ingestion pipelines (noise injection, orthogonal rotation, scaling, offset, fragmentation), and *VectorPin* [22], an Ed25519-over-canonical-bytes provenance protocol that pins each embedding to its source content and producing model so that any post-ingestion modification breaks signature verification — an analogous shape to the cryptographic audit-journal property in Section 9 but applied to the embedding layer rather than the action layer.

16.7 Structural Defenses for Content-Shape Vectors

The regex ceiling identified in Section 15 motivates structural-rather-than-substring approaches to content sanitization: typed agent-summary parameters whose admissible content is constrained by grammar; output-shape validators that enforce per-task expected-answer schemas; orchestrator-side prompt-construction patterns that isolate untrusted tool output from agent reasoning context; and downstream consensus or critic agents that act as task-correctness fences. Quantifying the marginal contribution of each move and identifying which combinations close the GPT-5-class outlier without overfitting is an open empirical question.

16.8 Multi-Implementation Conformance

The most important open question for the specification’s generality is whether independently-built OATS-compliant runtimes converge on the specified behavior. A target deliverable is at least one independent OATS-compliant implementation in a different language ecosystem, with the §12 conformance verification procedures re-run against it. The substrate-comparison methodology [24] provides a methodology template for cross-runtime comparison once a second compliant implementation exists.

17 Conclusion

OATS specifies what a zero-trust AI agent runtime should do to provide meaningful security properties for autonomous agent execution. The specification is grounded in three architectural convictions, with initial empirical results (Section 14.7) demonstrating implementation feasibility for five of seven core conformance requirements:

Allow-list over deny-list. Constraining what actions can be expressed reduces the attack surface compared to intercepting arbitrary actions and deciding which to block. The v11 ToolClad sweep [23] empirically validates this property at 100% bite-rate against hostile inputs across nine widely available hosted LLMs and eight argument-injection sub-shapes. The substrate-comparison sweep [24] strengthens the claim comparatively: at 0/560 escape on four pure-action vectors against Symbiont versus 88–98% pooled baseline rates on `python_baseline` and `python_sandboxed` with the same lures, structural-shape enforcement defends a threat class that OS isolation cannot reach.

Compile-time over runtime enforcement. Enforcing policy evaluation through the type system provides stronger structural assurance than testing it at runtime. Nine compile-fail tests in the reference implementation [27, §6.9] verify the `typestate` property on every CI run. The property holds within the `typestate`-governed code; it does not protect against bypasses that circumvent the loop entirely.

Structural independence over trust assumptions. Architecturally isolating the Gate from LLM influence reduces the risk that a compromised orchestration layer can influence policy evaluation. The v9 cloud-adversarial sweep refused 263 forbidden tool-call attempts across 874 runs with zero attempts reaching execution; the v12 stack-stripping ablation additionally surfaced a previously under-emphasized architectural finding: the action layer is two independent fences operating in series (Section 8.6), with the executor profile-of-one refusing 219 dispatches even when Cedar was disabled.

The substrate-comparison evidence newly available in v1.3.0 sharpens, not softens, the claims. It also identifies one bounded refinement: content-shape vectors against frontier models retain a *regex ceiling* that further marker-list expansion alone cannot close (Section 15). The architectural response to that ceiling — structural moves at the validator and at the orchestrator — is identified as research direction §16.7 rather than spec change.

The specification is informed by approximately nine months of autonomous operation in a production runtime (Symbiont [26]), including rebuilding a codebase using the runtime’s own agent infrastructure after a catastrophic loss event, and including a v1.14.0 security audit response release (May 2026) whose findings motivated several of the v1.3.0 SHOULD-level requirements. The empirical results reported in Section 14.7 substantiate a substantial subset of the specification’s claims; remaining items (context accumulation under load, Gate-influence probing, journal tamper-resistance evaluation, multi-implementation conformance, content-sanitization conformance under expanded adversarial corpora, controlled production case studies) are identified as future deliverables in Section 14.7 and Section 15.

By publishing this specification as an open standard, we aim to establish baseline requirements that enable comparable evaluation of runtime security approaches for autonomous agents. The goal is not to build OATS, but to define what an OATS-compliant system must do, enabling independent implementations to be measured against shared conformance criteria.

References

- [1] Amazon Web Services. Cedar: A language for defining permissions as policies. <https://aws.amazon.com/cedar/>

- [//www.cedarpolicy.com](http://www.cedarpolicy.com), 2023.
- [2] Anthropic. Model context protocol specification. <https://modelcontextprotocol.io>, 2024.
 - [3] Anton Chuvakin. Cloud CISO perspectives: How google secures AI agents. Google Cloud Blog, June 2025, 2025.
 - [4] Edoardo Debenedetti et al. AgentDojo: A dynamic environment to evaluate attacks and defenses for LLM agents. <https://arxiv.org/abs/2406.13352>, 2024. arXiv:2406.13352.
 - [5] H. Errico. Autonomous action runtime management (AARM): A system specification for securing AI-driven actions at runtime. <https://arxiv.org/abs/2602.09433>, 2026. arXiv:2602.09433v1.
 - [6] Sushant Gaire et al. Systematization of knowledge: Security and safety in the model context protocol ecosystem. <https://arxiv.org/abs/2512.08290>, 2025. arXiv:2512.08290.
 - [7] Kai Greshake et al. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *AISec Workshop at ACM CCS*, 2023.
 - [8] Norm Hardy. The confused deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988.
 - [9] Yupei Liu et al. Formalizing and benchmarking prompt injection attacks and defenses. In *USENIX Security Symposium*, 2024.
 - [10] Microsoft. Governance and security for AI agents across the organization. Cloud Adoption Framework, 2024.
 - [11] Mark S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006.
 - [12] National Institute of Standards and Technology. AI risk management framework (AI RMF 1.0), 2023.
 - [13] Open Policy Agent. OPA: Policy-based control for cloud native environments. <https://www.openpolicyagent.org>, 2024.
 - [14] OWASP Foundation. OWASP top 10 for large language model applications, 2024.
 - [15] Sander Perez et al. Ignore this title and HackAPrompt: Exposing systemic vulnerabilities of LLMs through a global prompt hacking competition. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
 - [16] Shaina Raza et al. TRiSM for agentic AI: A review of trust, risk, and security management in LLM-based agentic multi-agent systems. <https://arxiv.org/abs/2506.04133>, 2025. arXiv:2506.04133.
 - [17] Daniel Reber. The agentic AI security scoping matrix: A framework for securing autonomous AI systems. AWS Security Blog, November 2024, 2024.
 - [18] Yangjun Ruan et al. The emerged security and privacy of LLM agent: A survey with case studies. <https://arxiv.org/abs/2407.19354>, 2024. arXiv:2407.19354.

- [19] Hang Su et al. A survey on autonomy-induced security risks in large model-based agents. <https://arxiv.org/abs/2506.23844>, 2025. arXiv:2506.23844.
- [20] W3C. Trace context, level 2. <https://www.w3.org/TR/trace-context/>, 2024. W3C Recommendation.
- [21] Lei Wang, Chen Ma, Xueyang Feng, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024.
- [22] Jascha Wanger. VectorPin: Verifiable integrity for AI embedding stores. <https://github.com/ThirdKeyAI/VectorPin>, 2025. Apache-2.0.
- [23] Jascha Wanger. Making dangerous tool arguments inexpressible: A declarative contract format for LLM agents, 2026. Preprint v0.2, ThirdKey AI, April 2026.
- [24] Jascha Wanger. Substrate comparison: Empirical evaluation of structural enforcement against OS-isolation baselines, 2026. Preprint v0.2, ThirdKey AI, May 2026.
- [25] Jascha Wanger. Symbiont ORGA demo: Reproduction artifact for OATS empirical evaluations. <https://github.com/ThirdKeyAI/symbiont-orga-demo>, 2026. Accessed May 2026.
- [26] Jascha Wanger. Symbiont runtime architecture. <https://symbiont.dev>, 2026. ThirdKey AI.
- [27] Jascha Wanger. Typestate-enforced agent loops: Making policy gates unskippable at compile time, 2026. Preprint v0.5, ThirdKey AI, April 2026.
- [28] Jascha Wanger. VectorSmuggle: Steganographic exfiltration in embedding stores and a cryptographic provenance defense, 2026. Preprint, ThirdKey / Tarnover, LLC.
- [29] Qingyun Wu et al. Security of AI agents. <https://arxiv.org/abs/2406.08689>, 2024. arXiv:2406.08689.
- [30] Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [31] Qiusi Ye et al. ToolEmu: Identifying risky real-world agent failures with a language model emulator. In *International Conference on Learning Representations (ICLR)*, 2024.

A Future Directions for Adoption

For implementors. Build independent OATS-compliant runtimes across different language ecosystems. Multi-implementation conformance testing is the most important next step for validating the specification’s generality.

For evaluators. Apply the evaluation framework in Section 14 to existing and new agent runtimes. Comparative results across architectures would substantially strengthen or refine the claims made in this specification.

For researchers. Address open challenges in Section 16, particularly typestate portability across language ecosystems, formal verification of runtime properties, multi-agent trust coordination, and structural defenses for content-shape vectors (§16.7).

For the community. The specification is open and available at thirdkey.ai/oats. Feedback, critique, and competing proposals are welcome. The reference corpus and reproduction artifacts referenced in Section 14.7 are available at github.com/ThirdKeyAI/symbiont-orga-demo.

B Changes from v1.2.0

This revision integrates a third companion preprint (substrate comparison), a new content-sanitization requirement informed by reference-implementation hardening, identity-layer cryptographic-agility requirements, and a new limitation characterizing the content-shape ceiling against frontier models. The five-layer architecture, the three architectural convictions, the ORGA loop construction, and the core conformance requirements C1–C7 are unchanged.

- **Abstract.** Updated companion-preprint count (two → three); refreshed operational-experience figure (~eight months → ~nine months).
- **§1.3 Contributions.** Updated empirical-evaluation bullet to reflect three companion preprints, including comparative substrate evidence.
- **§6 Tool Contracts.** New §6.6 *Content Sanitization*: SHOULD-level requirement for stripping invisible Unicode and NFKC normalization on agent-influenced string fields, with a reference implementation pointer to `symbi-invis-strip` v0.3.0.
- **§7 Identity Layer.** New §7.5 *Cryptographic Agility and Algorithm Allowlisting*: SHOULD-level requirement that runtime verifiers refuse algorithm classes outside their declared allowlist; explicit guidance on JWT verifier configuration.
- **§8 Policy Enforcement Layer.** §8.1 clarified to specify that the runtime’s default *construction* MUST be fail-closed when no policy is wired (separating spec-level default-deny from operational fail-closed defaults). Empirical milestone referenced in §14.7.
- **§11 Inter-Agent Communication.** §11.5 added: distributed trace context (W3C *traceparent* recommended) as a SHOULD for cross-runtime tracing.
- **§12 Conformance Requirements.** New extended requirement E9: *Content Sanitization*. E1 reformulated to call out the algorithm-allowlist requirement explicitly.
- **§14.7 Initial Empirical Results.** New subsections summarizing the substrate-comparison sweep, the regex-ceiling / GPT-5 outlier characterization, and the v1.14.0 reference-implementation audit response.
- **§15 Limitations.** New bullets: *Content-shape ceiling on capable models*; *Answer-correctness orthogonal to structural fences*. Refreshed *Single reference implementation* to acknowledge comparative substrate evidence now exists, while still calling out the absence of independent OATS-compliant implementations.
- **§17 Conclusion.** Restatement of the three convictions updated with the substrate-comparison evidence.
- **References.** Substrate-comparison preprint added as a third companion paper; Zenodo DOIs added to the three companion papers; subsequent references renumbered.